Project Documentation Guidelines

Proposal, MVP Requirements, Technical Design, and AI-Assisted Development

Project-based Course 1105

Course adaptation draft, subject to change — 14 October 2026

1 Purpose and Learning Goals

Your project should address a real problem, produce a working deliverable, and demonstrate a complete user task. Documentation helps your team connect the problem to requirements, design decisions, implementation, and verification.

You may use AI to explore ideas, draft documents, compare options, and assist with implementation. Your team remains responsible for the accuracy of claims, the decisions you adopt, and the behavior of the final system. Every member should be able to explain their own contributions and the project's core workflow.

By completing these documents, you should learn to:

- formulate a concrete problem and support it with evidence;
- decompose the problem into a manageable minimum viable product (MVP);
- specify observable behavior and testable acceptance criteria;
- explain a simple technical design and its trade-offs;
- verify AI-assisted work and update documents as the project evolves.

2 Reference Workflow and Course Adaptation

The reference is KhazP's vibe-coding-prompt-template. It provides prompts that guide AI through research, PRD generation, technical design, and coding-assistant setup. These prompts are tools for producing documents; they are not themselves the documents to submit.

Reference component	Course use	Adaptation
Part 1: Deep Research	Problem investigation and proposal	Prioritize direct evidence and feasibility; omit irrelevant business analysis.
Part 2: MVP PRD	Main requirements document	Retain users, scope, workflows, business rules, and acceptance criteria.
Part 3: Tech Design	Brief implementation design	Focus on architecture, technology choices, data, and verification.
Part 4: Agent Setup	Supporting repository files	Keep instructions short, relevant, and consistent with the actual project.

The requirements below are a proposed course adaptation, not requirements imposed by the original repository. Follow these course requirements when the upstream prompts suggest additional material. Choose the beginner or learning-oriented interview path when appropriate. Do not add technologies or sections simply to fill a template.

3 Deliverables and Submission Stages

The following lengths are recommended targets, excluding references, interview notes, and prototype images. Concise tables and diagrams are welcome.

Stage	Deliverable	Expected scope
Project proposal	Proposal	1–2 pages: problem, users, evidence, solution, feasibility, roles, and milestones.
Before implementation	MVP PRD	3–5 pages: workflows, core features, rules, acceptance criteria, and exclusions.
After first prototype	Brief Tech Design	2–3 pages: system structure, stack choices, data, implementation approach, and checks.
During development	Agent instructions and decision record	Short, maintained files in the project repository.
Final submission	Updated documents and verification results	Actual implementation status, evidence, changes, and remaining limitations.

An initial technical sketch may be prepared before coding. Refine it after the first prototype rather than pretending that every design detail was known in advance. Record document dates and versions, and link related requirements, decisions, and tests using stable identifiers.

4 Project Proposal

Your proposal should explain why the project is worth doing and whether your team can complete it.

1. **Project summary.** Give the project name and a one-sentence description.
2. **Problem and target users.** Identify who experiences the problem, in what situation, and what currently goes wrong.
3. **Evidence.** Explain how you investigated the problem and what you learned. Separate observations from assumptions.
4. **Existing approaches.** Describe how users currently handle the problem, including manual workarounds, and identify a specific gap.
5. **Proposed solution and MVP value.** Describe the smallest useful solution and the complete user task it will support.
6. **Feasibility and risks.** State time, skills, data, equipment, access, budget, and external dependencies. Identify the most important uncertainty and how you will investigate it.
7. **Team and milestones.** Assign responsibilities and identify reviewable outputs for each milestone.

4.1 Evidence Expectations

Include at least one appropriate form of evidence: interviews with target users (for example, 3–5 people), observations of an existing workflow, requirements confirmed by an enterprise mentor, or relevant records/data demonstrating the problem. These are suggested methods; choose what fits the project and respect participants' privacy.

AI-generated personas and web summaries can help you prepare research, but do not replace evidence from the people or setting your project serves. Do not invent interview quotations, market sizes, user counts, or competitor limitations. Label an unverified claim as an assumption and explain how it could be checked. Include source links and access dates for externally sourced claims.

5 MVP Product Requirements Document

The PRD defines *what* the MVP must do, *who* it serves, and *why* its behavior matters.

5.1 Required Contents

1. **Overview and users.** Summarize the problem, target users, and intended value; link to proposal evidence.
2. **MVP scope.** Identify approximately 3–5 core features, adjusted to project complexity. Explain why each is necessary for the main user task.
3. **Out of scope.** Explicitly list deferred or excluded features and the reason for each exclusion.
4. **User journey and prototype.** Show the main end-to-end flow, key screens or interactions, and important decision points. For non-UI projects, show the equivalent input-to-output workflow.
5. **Functional requirements.** Assign stable IDs, such as FR-01. Describe expected behavior in clear, observable terms.
6. **Business rules and edge cases.** Define permissions, validation, conflicting actions, empty states, invalid inputs, and relevant failure behavior.
7. **Acceptance criteria.** State how each core requirement will be checked. Include relevant success and failure cases.
8. **Non-functional requirements.** Include project-relevant expectations for usability, performance, compatibility, security, and privacy. Specify realistic test conditions rather than copying arbitrary targets.
9. **Success measures and completion.** Explain how you will assess user value and when the MVP is ready for demonstration.
10. **Constraints, assumptions, and open questions.** State unresolved items and the next action or person responsible for resolving them.

5.2 Example: Campus Room Booking

ID	Requirement	Example acceptance criterion
FR-01	Create a booking	A valid booking is saved and remains visible after refreshing the page.
FR-02	Reject conflicts	A booking overlapping an existing booking for the same room is rejected with a clear explanation.
FR-03	Cancel a booking	An authorized cancellation releases the corresponding time slot.
FR-04	Validate times	An end time earlier than the start time produces an error and no booking is saved.

For FR-02, a more explicit scenario is:

> **Given** a room is booked from 14:00 to 15:00,
> **when** another student requests the same room from 14:30 to 15:30,
> **then** the system rejects the request, explains the conflict, and preserves the existing booking.

The team must define additional relevant rules, such as whether adjacent bookings are allowed. The example is illustrative, not a prescribed campus policy.

Distinguish **acceptance criteria** (whether the system behaves as specified) from **success measures** (whether it helps users). For example, rejecting overlapping bookings is an acceptance criterion; observing that target users can complete a booking without assistance is a user-value measure. Set the evaluation method and target before testing, and report actual results honestly.

6 Brief Technical Design

The technical design explains *how* your team will implement and verify the approved requirements. Include:

1. **System structure.** One simple diagram showing major components, their responsibilities, and important data flows.
2. **Technology choices.** List the selected tools and explain why they fit your team's skills, available infrastructure, and project constraints. Discuss a simpler alternative for major choices.
3. **Data model.** Identify principal entities, key fields, relationships, persistence, and access rules where relevant.
4. **Key implementation approaches.** Explain how the main requirements map to modules or components. Describe important interfaces when needed; an exhaustive API specification is unnecessary for a small prototype.
5. **Failure handling.** Explain what happens when inputs are invalid, a service fails, or a required resource is unavailable.
6. **Run and verification instructions.** Provide prerequisites and exact commands or steps to install, run, and check the project. Mark proposed commands as unverified until executed.
7. **Deployment and costs, if applicable.** State where the project will run, who controls it, required configuration, and any expected charges or limits. Local execution is acceptable when it meets the agreed project requirements.

For every major component, be ready to answer: Which requirement does it support? Why is it needed? Could a simpler approach work? What happens if it fails?

Do not add authentication, databases, payments, microservices, public hosting, or AI features unless your requirements justify them. Advanced scaling plans, monetization, multi-agent orchestration, and detailed cloud comparisons are optional extensions, not default first-year deliverables. Mark genuinely inapplicable items as "Not applicable" with a brief explanation.

7 AI-Assisted Development and Supporting Records

7.1 Development Assistance Versus Product AI

Using AI to write code does not mean the product itself needs AI. If your product includes an AI feature, additionally state its purpose, permitted data access, expected output, failure behavior, evaluation examples, and cost limits. Identify any actions that require user confirmation. Keep this proportional to the actual feature.

7.2 Repository Instructions

Maintain a concise `AGENTS.md`, or the equivalent instruction file supported by your chosen tool, containing project constraints, links to the authoritative PRD and technical design, useful commands, and verification expectations. Record changing progress separately. Supporting files generated by Part 4 should be kept only when useful; do not maintain duplicate summaries that contradict the main documents.

7.3 Decision and Verification Record

Full chat transcripts are not required. Record a few representative decisions showing how your team evaluated AI suggestions:

AI suggestion or claim	Team response	Reason and verification
Add user login	Deferred	The current prototype validates booking behavior using demonstration roles.
Use shared database storage	Accepted	Shared persistence is required; saving and retrieving bookings was checked.
Booking feature is complete	Rejected, then corrected	Refreshing lost the booking; persistence was added and checked again.

For important bugs, record the symptom, expected behavior, investigation, fix, and a check that demonstrates the original problem is resolved. Students should explain what they verified, not simply report that an AI assistant declared success.

8 Final Submission and Assessment

Submit updated documents with the code repository and demonstration materials. For every core requirement, report its final status (implemented, changed, removed, or incomplete), relevant verification evidence, and any limitation. Explain scope changes rather than silently rewriting the original plan.

A simple traceability table is sufficient:

Requirement	Component or module	Verification evidence	Final status
FR-02	Booking validation	Overlap scenario; test result or demo reference	Implemented / changed / incomplete

Assessment should emphasize the following dimensions. No numerical weights are prescribed in this draft.

- **Problem quality:** a specific problem supported by credible evidence.
- **Scope quality:** a useful, achievable MVP with justified exclusions.
- **Requirement quality:** clear behavior, rules, and testable acceptance criteria.
- **Design understanding:** simple, justified choices that the team can explain.
- **Implementation consistency:** the demonstrated system matches the documented requirements, or changes are explained.
- **Verification and AI judgment:** meaningful checks, honest limitations, and evidence of reviewing AI output.

Document length, technical jargon, and the number of technologies used do not by themselves demonstrate engineering quality. Short oral questions or a small change request during review can help establish whether the team understands its work.

This document adapts the workflow for a first-year educational setting. It is not an official document of the repository author. Reference pages were reviewed on 14 October 2026; the links above point to the evolving main branch.